

Scalable Technologies for Data Analysis

Machine Learning

Davide Carneiro

Mestrado em Engenharia Informática

2025/2026

Machine Learning

How to implement a ML pipeline in a Data Platform

- By the end of this class you should be able to
 - Explain where ML fits in a modern data platform
 - Design features and labels without temporal leakage
 - Build reproducible training and evaluation pipelines
 - Evaluate models against meaningful baselines and metrics
 - Compare batch scoring and online serving trade-offs
 - Describe a minimal MLOps lifecycle
 - Integrate ML workflows into Flyte tasks and pipelines

The plan

1. Intro to modern Data Platforms and scalability
2. Contract-Driven Design and Data Flow Modeling
3. Storage + table formats
4. Processing patterns + optimization
5. Orchestration + production reliability
6. Serving: SQL/APIs/semantic layer
7. Quality + observability + cost
8. Incremental + streaming
9. ML integration 
10. Governance + evolution

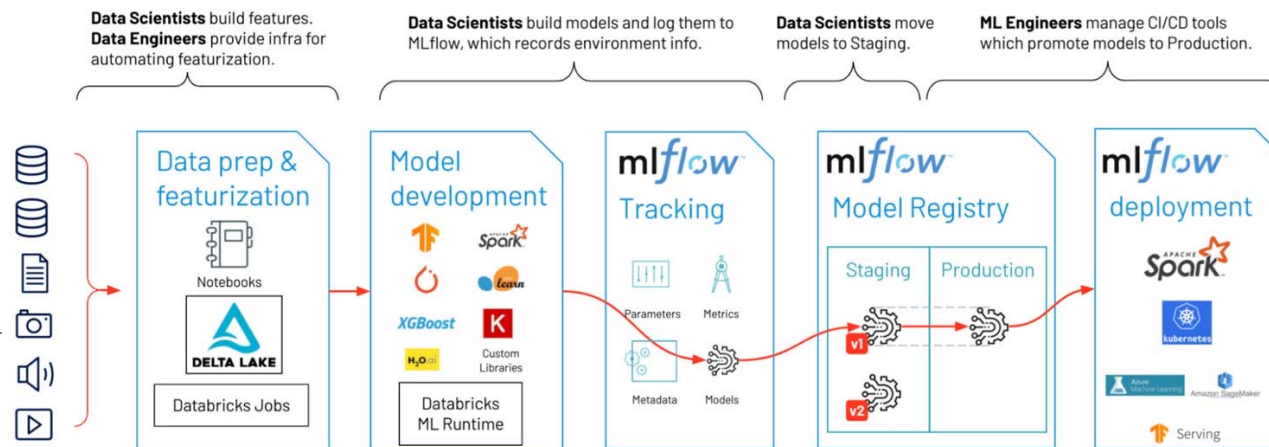
Previously...

- In the previous classes, we...
 - Designed data platforms around data products, contracts, and Bronze / Silver / Gold layers
 - Learned how to store and organize analytical data efficiently using object storage, Parquet, partitioning, and Iceberg
 - Built scalable transformations and reliable workflows with Trino and Flyte
 - Discussed serving, quality, observability, idempotency, retries, and backfills as foundations for production-grade pipelines
- Today we...
 - Learn how to leverage data products to train ML Models
 - Discuss how typical ML pipelines can be implemented in the context of a Data Platform

Introduction

ML in a data platform

- In a modern organization, ML is not a separate magical layer: it is another component of the data platform
- ML models **consume data** produced by pipelines, and they also **produce new outputs** that other people, systems, dashboards, or applications may consume
- In practice, this means that ML should not be treated as something external to the platform. It should follow the same principles we already discussed for data products, contracts, orchestration, quality, observability, and serving
 - But it caters to a specific set of users...



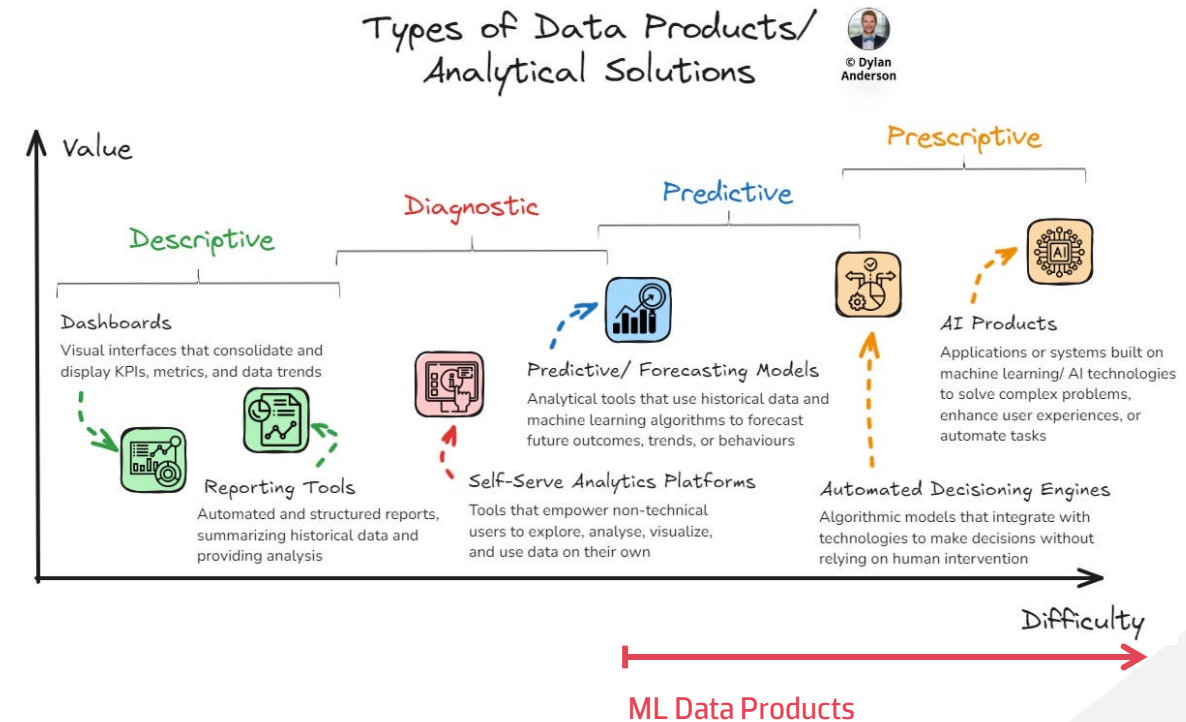
ML consumes *trusted* data products

- ML models depend on reliable data
- In a modern data platform, training datasets and feature tables are usually built from Silver or Gold data products, not directly from raw data
- This matters because models depend on assumptions about:
 - What each row represents
 - Which keys uniquely identify entities or events
 - How timestamps are defined
 - Which records are included or excluded
 - How missing values, duplicates, and late data are handled
 - How fresh the data is expected to be
- If these assumptions are unclear, the model may still train successfully, but its outputs may be unreliable
- If we decide on our own, we might be influencing the resulting model in undesirable ways



ML produces new data products

- ML does not only consume data: it also produces data
- Typical ML outputs include:
 - Churn probabilities
 - Fraud risk scores
 - Customer segments
 - Product recommendations
 - Demand forecasts
 - Anomaly alerts
 - Ranking scores
 - Next-best-action suggestions
- These outputs are not just temporary model results. In a platform context, they often become inputs to dashboards, campaigns, APIs, operational systems, or downstream pipelines
- Therefore, ML outputs should also be treated as data products. As such, they need:
 - Clear consumers
 - Stable schemas
 - Documented meaning
 - Ownership
 - Freshness expectations
 - Versioning
 - Monitoring
 - Serving strategy
- The key message is predictions are data products too



ML needs production-grade properties

- In ML, training a model is only one part of the problem
- If ML is part of the data platform, then ML pipelines need the same production-grade properties as other data pipelines
 - They need **contracts**
 - They need **reproducibility**
 - They need **validation**
 - They need **observability**
 - They need **versioning**
 - They need **safe serving**

Why?

ML needs production-grade properties

- In ML, training a model is only one part of the problem
- If ML is part of the data platform, then ML pipelines need the same production-grade properties as other data pipelines
 - They need **contracts**, so consumers know what each feature, label, score, or prediction means
 - They need **reproducibility**, so we can understand which data, code, parameters, and model version produced a result
 - They need **validation**, so bad training data, broken features, or invalid predictions do not silently reach consumers
 - They need **observability**, so we can detect failures, drift, performance degradation, freshness problems, and abnormal prediction behavior
 - They need **versioning**, so models, features, data snapshots, and outputs can evolve safely
 - They need **safe serving**, because predictions may affect business decisions, users, or automated actions
- ML integration is not just about training models. It is about **operating reliable predictive data products**



ML in a Data Platform

Where it fits

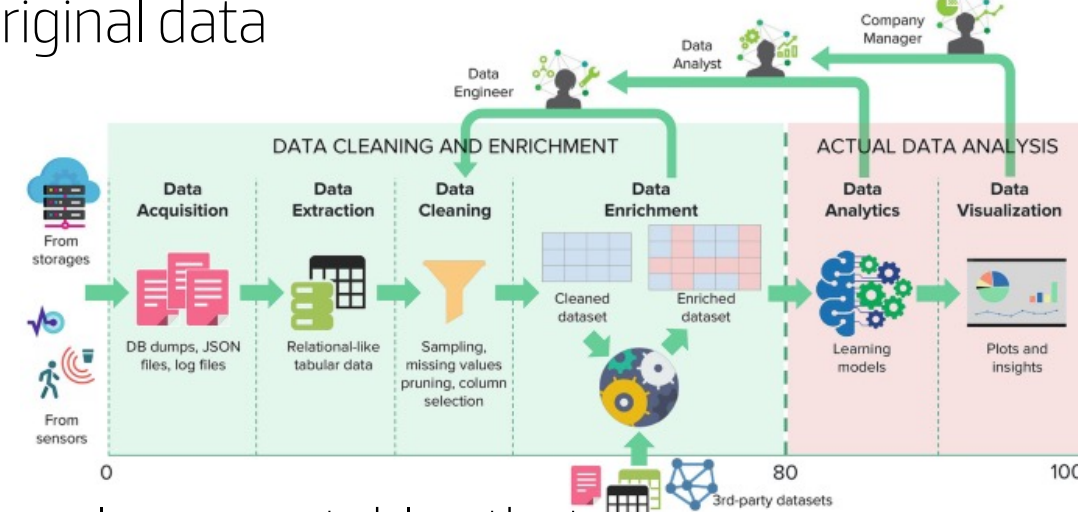


Where ML fits in the data platform

- ML can appear in different places inside a data platform
 - Sometimes it **enriches** existing data
 - Sometimes it **predicts** future outcomes
 - Sometimes it **ranks** options
 - Sometimes it **detects** unusual patterns
- The important point is that ML should not be seen only as “model training”. In a production data platform, ML is part of a larger flow: data is collected, cleaned, modeled, transformed into features, used for training or scoring, and then served back to consumers
- So, ML is not one single use case. It is **a set of patterns** that can be integrated into the platform in different ways

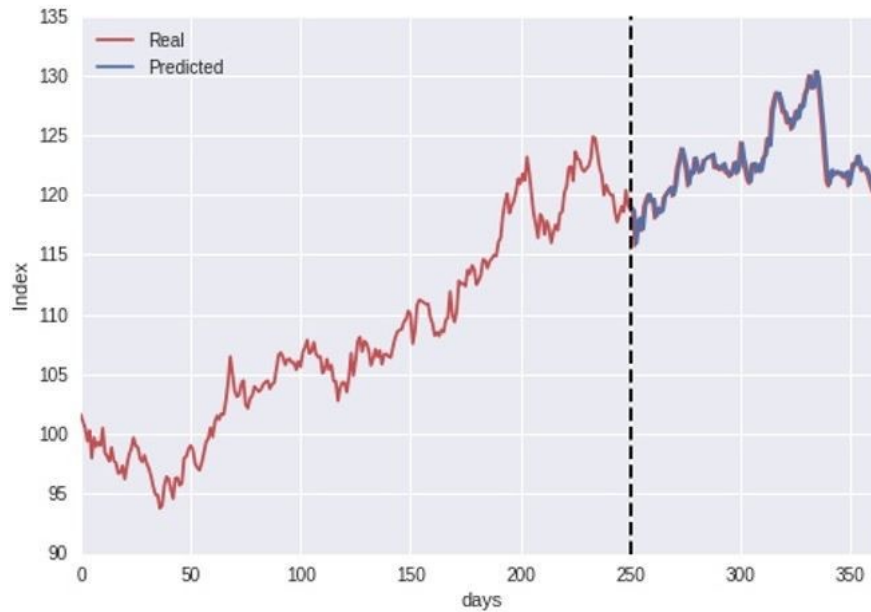
Pattern 1: enrichment

- In the context of data and data platforms, enrichment means adding inferred attributes to existing entities
- The goal is not necessarily to predict a future event. Instead, the model adds extra information that was not directly available in the original data
- Examples
 - Assigning a customer segment
 - Classifying a product category
 - Estimating a risk tier
 - Detecting the topic of a support ticket
 - Identifying the language or sentiment of a text
 - Grouping users into behavioral profiles
- In platform terms, enrichment often produces new columns or tables that downstream consumers can reuse
- It turns raw or cleaned data into more informative data products



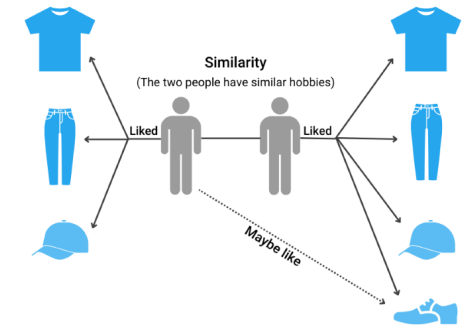
Pattern 2: prediction

- Prediction means estimating a future value or event
- The model is built using historical data available in the lakehouse, to then estimate something that has not happened yet or is not yet known
- Examples
 - Probability that a customer will churn
 - Expected demand for a product next week
 - Probability that a delivery will be delayed
 - Estimated lifetime value of a customer
 - Probability that a transaction is fraudulent
 - Expected sales for a store tomorrow
- Prediction usually requires a clear definition of
 - The entity being predicted
 - The observation time
 - The features available at that time
 - The future outcome being predicted
 - The prediction horizon

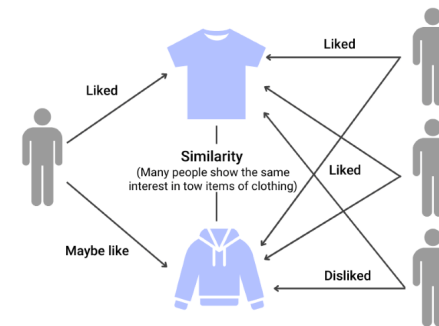


Pattern 3: ranking and recommendation

- Ranking means ordering items according to expected relevance, value, or utility. It is the type of task that usually supports recommender systems, such as in online stores or in social media
- Unlike simple prediction, ranking usually compares multiple possible options for the same context
 - Ranking is not only about predicting a score. It is about producing an ordered decision-ready output
- Examples
 - Top products to recommend to a user
 - Next best action for a customer
 - Most relevant search results
 - Highest-risk transactions for manual review
 - Most promising leads for a sales team
 - Best content items to show on a homepage
- In a data platform, ranking outputs may be consumed by APIs, applications, campaign tools, or analytical dashboards



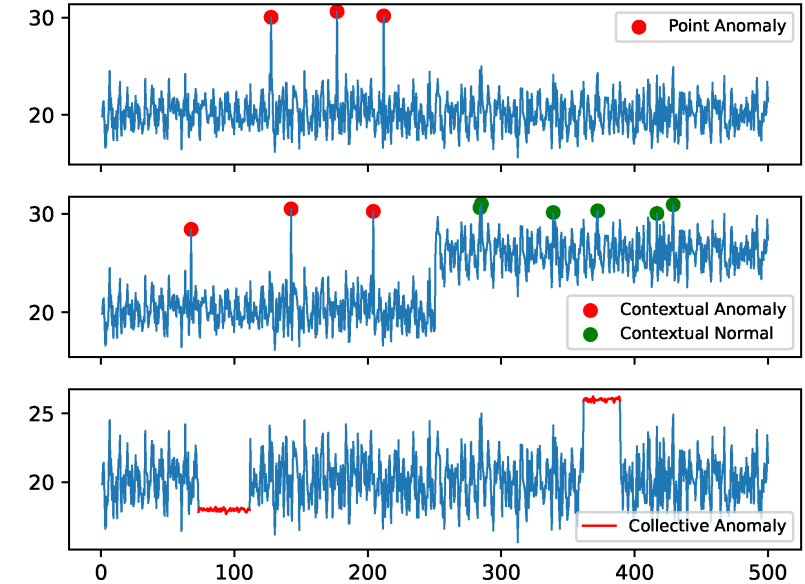
User-based



Item-based

Pattern 4: anomaly detection

- Anomaly detection means identifying unusual behavior or suspicious patterns
- The goal is to detect cases that are different from what is expected
- Examples
 - Fraud alerts
 - Unusual login behavior
 - Sudden drops in data volume
 - Unexpected changes in revenue
 - Abnormal sensor readings
 - Data quality anomalies
 - Operational incidents
- Anomaly detection is especially relevant because it can **apply both to business data and to the data platform itself**. In that sense, it can support business decisions and platform observability
- For example, anomaly detection can identify
 - A customer with unusual transaction behavior
 - A pipeline with an unexpected row count drop
 - A metric that suddenly changes outside its normal range
 - A scoring job producing abnormal prediction distributions



ML outputs are platform outputs

- All four patterns produce outputs that other systems or people may consume
- These outputs may be enriched attributes, probabilities, forecasts, rankings, alerts, segments, scores or recommendations
- In a production platform, these outputs should not remain hidden inside notebooks, temporary files, or isolated scripts: they should be modeled as data products
- This means defining
 - Who consumes the output
 - What each row represents
 - What each score or label means
 - How fresh the output must be
 - How the output was produced
 - Which model version produced it
 - How quality is validated
 - How changes are monitored
- As a general rule, if an ML output influences a decision, it should have a contract, lineage, and monitoring

Different ML patterns, different platform requirements

- Different ML patterns place different requirements on the data platform.
- The platform does not need to support every ML use case in the same way
 - Some ML outputs can be computed once per day and served as a table
 - Others must be computed immediately when new data arrives
- This means that before designing the ML pipeline, we need to understand the operational nature of the ML use case

ML pattern	Typical output	Freshness requirement	Typical processing mode	Typical serving mode	Main platform challenge
Enrichment	New attributes added to existing entities	Low to medium	Usually batch	Tables, marts, APIs	Keeping enriched attributes versioned, documented, and reusable
Prediction	Probability, forecast, or expected value	Low to high, depending on use case	Often batch, sometimes online	Batch tables or prediction API	Avoiding leakage and keeping features consistent between training and scoring
Ranking / recommendation	Ordered list of candidates	Medium to very high	Batch, online, or hybrid	API, cache, materialized recommendation table	Low-latency serving and frequent refresh of user/item context
Anomaly detection	Alert, anomaly score, abnormal event flag	Often high or very high	Streaming or near-real-time when action is urgent	Alerts, monitoring systems, operational dashboards	Detecting anomalies quickly while controlling false positives

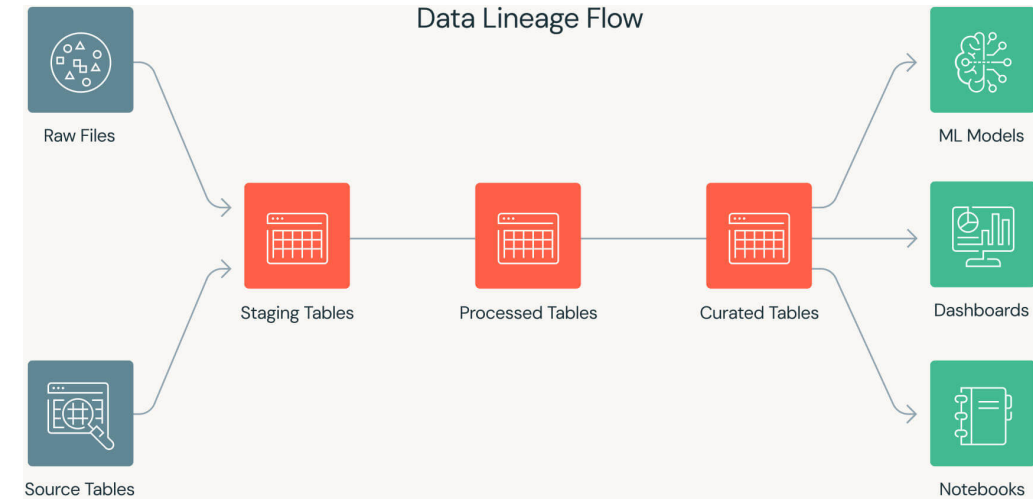
ML outputs as data products

- ML is not only about producing a model: it is about producing reliable outputs that other consumers can use
- In a data platform, **the model itself is only one part of the product**. The product may include feature tables, training datasets, model artifacts, prediction tables, APIs, monitoring outputs, and documentation
- An ML product may take different forms
 - `gold.customer_churn_features`
 - `ml.churn_model_v3`
 - `gold.customer_churn_scores_daily`
 - `serving.customer_churn_api`
- Each artifact has a different role
 - The feature table prepares reusable inputs
 - The model artifact stores trained logic
 - The scoring table publishes predictions
 - The API exposes predictions to applications
- A complete ML use case is usually **a family of *connected* products**, not a single model file

What do you think these products are ?

Lineage matters for ML

- For every prediction, we should be able to answer
 - Which data produced this prediction?
 - Which feature definitions were used?
 - Which model version was used?
 - Which code version generated the score?
 - When was the prediction produced?
 - Was the output validated before publication?
- This is essential for debugging, auditing, monitoring, and trust
- Nowadays, we can combine tools like MinIO, Flyte and MLFlow to be able to completely log all this information
 - This is essential for auditing AI pipelines, especially to comply with the requirements of the AI Act
 - But also simply for matters of transparency and debugging
- The key message is predictions must be traceable from source data to final served output



Defining the ML problem

From a Data Platform perspective

Defining the ML problem

- This is one of the most important parts of ML in a data platform
- Before choosing an algorithm, we must define the data problem correctly
- In ML, many failures do not come from the model itself. They come from unclear definitions of features, labels, time, grain, and validation
- A model can train successfully and still be wrong if the training data does not represent the real prediction scenario
- So, good ML starts before the algorithm. It starts with correct data design
- To define an ML problem, we need a few basic concepts
 - An **entity** is the thing we are making predictions about. (e.g. a customer, an order, a product, a transaction, a session, a store, a sensor)
 - A **feature** is an input variable available at prediction time
 - A **label** is the target outcome used during training
 - An **observation time** is the moment at which the prediction is made
 - A **prediction horizon** is the future period we want to predict

Features must match the prediction moment

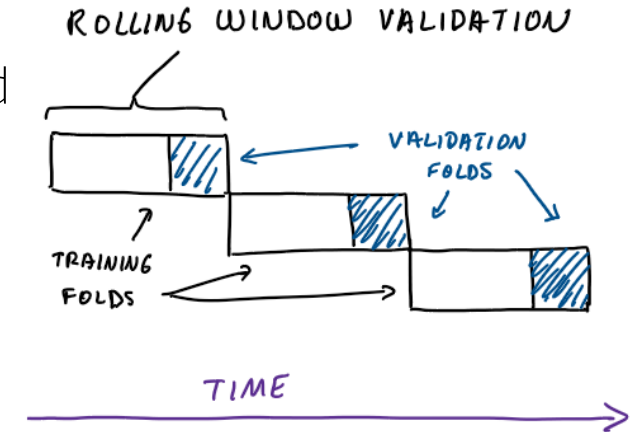
- A feature is valid only if it would be available at the moment of prediction
- For example, suppose we predict customer churn on 2026-03-01
- Valid features
 - `number_of_orders_last_30_days`
 - `total_spent_last_90_days`
 - `days_since_last_order`
 - `support_tickets_last_60_days`
- Invalid features
 - `number_of_orders_next_30_days`
 - `refunds_after_2026_03_01`
 - `customer_status_updated_after_prediction`
 - `final_lifetime_value`
- The invalid features may be strongly correlated with churn, but they **use information from the future**
- That makes the model look better during training than it will be in production
- So, a feature is not valid because it is predictive: it is valid only if it is available at prediction time

Leakage

- Leakage is any information used during training that would not be available when the model is used in production
- It is dangerous because it creates an illusion of performance
 - The model may look excellent during validation, but fail when deployed
- This happens because the model has learned from information that **belongs to the future**, or from information that **indirectly reveals the label**
- Leakage is one of the most common reasons why offline ML performance does not survive production
- Common leakage patterns include
 - Using future events as features
 - Computing aggregates over the full dataset instead of only past data
 - Using random train/test splits for temporal problems
 - Joining with tables that were corrected after the prediction time
 - Using columns that directly or indirectly reveal the label
 - Normalizing data using statistics computed over the full dataset
 - Selecting features after looking at test performance
 - Including post-outcome statuses, cancellations, refunds, or final states

Leakage through wrong splits

- Random splits are often wrong for temporal prediction problems: a random split may place records from the same customer, product, or time period in both training and test data
- This allows the model to learn patterns from the future or from highly similar examples
- For time-dependent problems, the correct structure is usually:
 - Train period → Validation period → Test period → Production scoring period
- Example
 - Train: January to June
 - Validation: July to August
 - September to October
 - Production scoring: November onward



- If the model is supposed to predict the future, validation must simulate the future

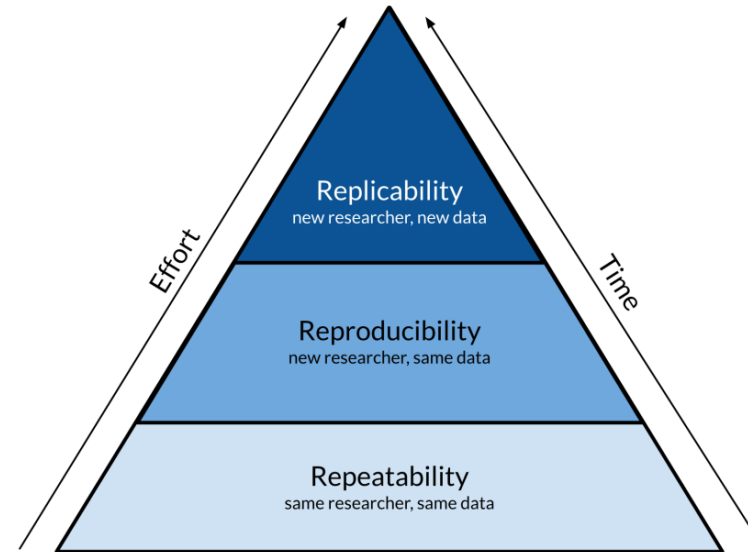
Correct temporal validation

- A good validation setup should answer this question: If we had trained the model in the past, how well would it have performed in the future?
- This means
 - Training data must come before validation data
 - Validation data must come before test data
 - Feature computation must respect observation time
 - Labels must be computed only after the prediction horizon
 - Production scoring must follow the same feature logic as training
- The goal is not only to estimate model accuracy: the goal is to simulate the real production situation as closely as possible.

Reproducible training

Reproducible training

- A model is only reproducible if we can reproduce everything that produced it
- This includes
 - The data
 - The code
 - The hyperparameters
 - The dependencies
 - The execution environment
 - The artifacts
 - The workflow execution metadata
- Reproducibility is not only about setting a random seed: it is about being able to explain and recreate the full training run



The reproducibility problem

- A common anti-pattern in reproducibility is: *"I trained the model in a notebook last week"*
- This is not enough as important questions remain unanswered:
 - Which dataset was used?
 - Which partitions were included?
 - Which feature definitions were used?
 - Which code version produced the model?
 - Which Python packages were installed?
 - Which hyperparameters were selected?
 - Which random seed was used?
 - Which evaluation results were produced?
 - Where is the model artifact stored?
- A better statement is: *"This model was trained by workflow X, version Y, using data snapshot Z, parameters P, and produced artifacts A."*
- A model that cannot be traced cannot be trusted (and should not be used!)

Enabling reproducibility

1. Know exactly which data was used (**data versioning**). For a reproducible training run, we should record dataset name, table name, data layer, partition range, extraction time, table snapshot, contract version, feature version...
2. Know exactly which code produced the model (**code versioning**). For a reproducible training run, we should record Git commit, branch or tag, package version, workflow version, task version, SQL query version, feature generation code version...
3. Reproduce the environment (**environment versioning**). The same code may behave differently if the environment changes. We should record container image, Python version, package versions, dependency lock file, operating system base image, system libraries, hardware assumptions (when relevant)...
4. Record the training configuration (**training versioning**). This includes hyperparameters, random seeds, selected features, label definition, train/validation/test split rules, prediction horizon, preprocessing options, threshold selection rules...
5. A training run should produce **artifacts** that can be inspected, reused, and audited. Typical artifacts include model file, preprocessing pipeline, feature schema, evaluation metrics, confusion matrix, calibration plot, feature importance, evaluation report, prediction samples, training logs...
6. The final requirement is **execution metadata**. For each training run, we should know workflow run ID, task IDs, start time, end time, input parameters, output locations, user or service account, execution status, logs, retries, resource usage...

Minimum reproducibility checklist

- As a minimum, we should try to record the following elements for each ML pipeline

Area	What to record
Data version	Dataset name, partition range, table snapshot, contract version
Code version	Git commit, package version, workflow version
Environment	Container image, Python version, dependency lock file
Training parameters	Hyperparameters, random seeds, feature list, label definition
Artifacts	Model file, metrics, plots, feature schema, evaluation report
Execution metadata	Workflow run ID, timestamps, inputs, outputs, logs

Reproducible training as a platform capability

- Reproducibility should not depend on individual discipline alone. The platform should make reproducibility the default
- This means
 - Training runs are executed through workflows
 - Workflows use versioned tasks
 - Tasks run inside containers
 - Inputs and outputs are explicitly declared
 - Data snapshots are recorded
 - Parameters are stored
 - Artifacts are persisted
 - Logs and metadata are available after execution
- This connects directly with the broader data platform principles already discussed in TEAD: orchestration, versioning, observability, idempotency, and auditability
- Reproducible ML is not only an ML concern: it is a data platform concern

Serving predictions

Batch scoring vs online serving

Serving predictions

- Training a model is not the end of the ML lifecycle: once a model exists, we need to decide how its predictions will be made available to consumers
- There are two common serving patterns
 - Batch scoring
 - Online serving
- The right choice depends on the decision that consumes the prediction
- So, serving predictions is a platform design decision, not only an ML decision

Batch scoring

- Batch scoring means periodically computing predictions for many entities and storing the results as a table
 - Example: `gold.customer_churn_scores_daily`
- Each run scores a population of entities, such as all active customers, all products, all stores, or all open orders
- The output is then consumed later by dashboards, reports, campaigns, APIs, or downstream pipelines
- So, batch scoring turns model predictions into a reusable data product
- Batch scoring is usually a good fit when predictions support decisions that do not need to happen immediately
 - Daily or weekly business decisions
 - Marketing campaigns
 - Customer retention lists
 - Dashboards
 - Financial or operational reports
 - Demand forecasts
 - Risk monitoring
 - Large-scale scoring over many entities
- It is usually **cheaper and easier to operate** because predictions are computed on a schedule, not on every user request
 - We can often even delay the computing dynamically, according to the load on the cluster
- It is easier to validate before publication because the platform can check the scoring output before exposing it to consumers

Online serving

- Online serving means computing predictions on demand, usually through an API
- Instead of precomputing predictions for all entities, an application sends a request when it needs a prediction
 - Example: `POST /predict/churn/customer_id=123`
- Online serving is a good fit when the prediction must use fresh context or support immediate action
 - User-facing recommendations
 - Fraud detection at transaction time
 - Real-time personalization
 - Search ranking
 - Dynamic pricing
 - Next-best-action systems
 - Operational decisions requiring low latency
- In these cases, the prediction cannot always be computed hours earlier: the model needs to respond when the event happens or when the user interacts with the system

The serving decision framework

- To choose between batch scoring and online serving, ask:
 - When is the prediction needed?
 - How fresh must the input data be?
 - How quickly does the prediction become stale?
 - Who consumes the prediction?
 - What latency does the consumer expect?
 - How many predictions are needed?
 - Can the output be validated before publication?
 - What happens if the prediction is wrong or unavailable?
- The answer is not always one or the other: many systems use a **hybrid approach**

Hybrid serving

- In many real systems, batch and online serving are combined
- Example 1
 - Batch pipeline: precompute candidate recommendations every night
 - Online service: rerank candidates using current session context
- Example 2
 - Batch pipeline: compute daily churn risk for all customers
 - Online service: retrieve the latest score when a customer opens the app
- Hybrid serving can reduce online complexity while still supporting relatively fresh and useful predictions

MVMLOps

Minimum Viable MLOps

Minimum viable MLOps

- MLOps is the set of practices that makes ML outputs reproducible, governable, deployable, monitorable, and safe to change
 - The goal is not to create unnecessary bureaucracy around ML
 - The goal is to make sure that models and predictions can be trusted once they become part of the data platform
- A model that works once in a notebook is not enough. A production ML system must answer questions such as
 - Which model is currently in production?
 - Which data was it trained on?
 - Which features does it use?
 - How was it evaluated?
 - Where are its artifacts stored?
 - What predictions did it produce?
 - Is it still behaving as expected?
 - Can we roll it back safely?
- MLOps is what turns model experimentation into reliable platform operation. And Data Platforms can and should be a central enabler

Why “minimum viable” MLOps?

- MLOps can become very complex. In large organizations, it may include feature stores, automated retraining, model approval workflows, canary deployments, shadow deployments, real-time monitoring, advanced drift detection, automated rollback...
- More than knowing all the technologies and approaches, for this course, the goal is to understand the minimum viable set of practices that makes ML safe enough to integrate into a data platform. At minimum, we need
 - Experiment tracking
 - Model registry
 - Artifact storage
 - Prediction logging
 - Monitoring
 - Rollback strategy
- These components create the basic operational foundation for ML
- So, in our case, minimum viable MLOps means starting with the practices that make ML traceable, observable, and recoverable

Component 1: experiment tracking

- Experiment tracking records what happened during training (full list in the reproducibility slides!). Without experiment tracking, model development becomes difficult to trust
- Common problems include
 - Not knowing which run produced the best model
 - Comparing models trained on different data without realizing it
 - Losing the hyperparameters that produced a result
 - Being unable to reproduce an evaluation
 - Selecting a model based on incomplete evidence
 - Deploying a model without knowing its training context
- Experiment tracking creates a structured memory of model development
- It allows us to move from: *"This model seems better"* to *"This model performs better on the same test period, using the same label definition, and its artifacts are linked to this exact training run."*
- If we do not track experiments, model selection becomes guesswork

Component 1: experiment tracking

mlflow 3.0.0rc3

Experiments Models Prompts

GitHub Docs

Experiments

Search experiments

☐ Default

☒ MLflow 3.0 Tracking Example

MLflow 3.0 Tracking Example

Provide Feedback

Add Description

Share

Runs Models Experimental Evaluation Traces

metrics.rmse >= 0.8

Datasets

Sort: Creation time

Columns

Model attributes			Model attributes		Dataset: train (#1f1c13b5)	Parameters
Model name	Status	Created ↓	Source run	Dataset	accuracy	activation
torch-iris-100	Ready	26 seconds ago	popular-snake-452	train (#1f1c13b5)	0.9833333333333333	ReLU
torch-iris-90	Ready	29 seconds ago	popular-snake-452	train (#1f1c13b5)	0.9833333333333333	ReLU
torch-iris-80	Ready	31 seconds ago	popular-snake-452	train (#1f1c13b5)	0.9833333333333333	ReLU
torch-iris-70	Ready	33 seconds ago	popular-snake-452	train (#1f1c13b5)	0.9833333333333333	ReLU
torch-iris-60	Ready	35 seconds ago	popular-snake-452	train (#1f1c13b5)	0.9833333333333333	ReLU
torch-iris-50	Ready	37 seconds ago	popular-snake-452	train (#1f1c13b5)	0.9833333333333333	ReLU
torch-iris-40	Ready	39 seconds ago	popular-snake-452	train (#1f1c13b5)	0.975	ReLU
torch-iris-30	Ready	41 seconds ago	popular-snake-452	train (#1f1c13b5)	0.9416666666666667	ReLU
torch-iris-20	Ready	44 seconds ago	popular-snake-452	train (#1f1c13b5)	0.675	ReLU
torch-iris-10	Ready	46 seconds ago	popular-snake-452	train (#1f1c13b5)	0.6583333333333333	ReLU
torch-iris-0	Ready	48 seconds ago	popular-snake-452	train (#1f1c13b5)	0.325	ReLU

Component 2: model registry

- A model registry keeps track of model versions and their lifecycle state
- A model may move through different stages (Candidate → Staging → Production → Archived), and each stage has a different meaning
 - A **candidate** model was trained and evaluated, but is not ready for deployment
 - A **staging** model is being reviewed, tested, or prepared for production
 - A **production** model is the current model used to generate predictions
 - An **archived** model is no longer active, but is kept for traceability, auditability, or rollback
- Essentially, the model registry answers a simple but critical question:
which model version is allowed to be used?

Component 2: model registry

mlflow2.10.0

ExperimentsModels

GitHub

Docs

Registered Models

Create Model

Filter registered models by name o...

Name ↕	Latest version	Aliased versions	Created by	Last modified	Tags
iris_model_dev	Version 17			2023-09-25 12:50:...	—
iris_model_prod	Version 11	@ champion : Version 11 +3		2023-10-26 17:10:...	—
iris_model_staging	Version 11			2023-09-25 12:46:...	—
iris_model_testing	Version 1			2023-09-27 13:17:...	—
mnist_model_dev	Version 12				
mnist_model_prod	Version 8	@ challenger : Version 8 +1			
mnist_model_staging	Version 8				

New model registry UI

Registered Models >

mnist_model_prod

Created Time: 2023-09-25 12:10:09

Last Modified: 2024-01-19 10:35:06

> Description

Edit

> Tags

▼ Versions

Compare

Version	Registered at ↕	Created by	Tags	Aliases	Description
✓ Version 8	2023-09-25 12:38:44		validation_status: pending	@ challenger	Uses decision tree
✓ Version 7	2023-09-25 12:38:41		validation_status: approved	@ champion	Uses a CNN based approach
✓ Version 6	2023-09-25 12:38:38		validation_status: approved	Add	
✓ Version 5	2023-09-25 12:38:35		Add	Add	
✓ Version 4	2023-09-25 12:38:32		Add	Add	

New model registry UI

Component 3: artifact storage

- Training produces more than a model name: it produces artifacts that must be stored, versioned, and linked to the training run
- Typical artifacts include:
 - Model file
 - Preprocessing pipeline
 - Encoders
 - Scalers
 - Feature schema
 - Evaluation report
 - Metrics file
 - Plots
 - Calibration report
 - Threshold configuration
 - Dependency metadata
- These artifacts should not be stored randomly on a developer machine: they should be persisted in a controlled location, such as an artifact store or object storage bucket

Component 3: artifact storage

Azure AI | Machine Learning Studio

contosobami > Contoso > Models > credit_defaults_model:3

credit_defaults_model:3

Details Versions **Artifacts** Endpoints Jobs Data Feature sets Responsible AI Explanations (preview) Fairness (preview)

Refresh Download all

credit_defaults_model

- conda.yaml
- MLmodel**
- model.pkl
- python_env.yaml
- requirements.txt

```
1 artifact_path: credit_defaults_model
2 flavors:
3   python_function:
4     env:
5       conda: conda.yaml
6       virtualenv: python_env.yaml
7     loader_module: mlflow.sklearn
8     model_path: model.pkl
9     predict_fn: predict
10    python_version: 3.8.19
11  sklearn:
12    code: null
13    pickled_model: model.pkl
14    serialization_format: cloudpickle
15    sklearn_version: 1.0.2
16 mlflow_version: 2.8.0
17 model_size_bytes: 121220
18 model_uuid: 35a82588e7cc4a01a0d4a98a3bd67c62
19 run_id: boring_leek_r03yywh7q
20 utc_time_created: '2024-09-03 18:11:02.915049'
```

mlflow 2.19.0 Experiments Models

/mlflow-pytorch-quickstart_1 >

selective-lamb-593

Overview Model metrics System metrics **Artifacts**

- data
 - MLmodel
 - conda.yaml
 - python_env.yaml
 - requirements.txt
 - model_summary.txt

model

Path: s3://mlflow-bucket/2/7410242a778346a690aad5aee9cd9f34/artifacts/model

MLflow Model

The code snippets below demonstrate how to make predictions using the logged model. You can also [register it to the model registry](#) to version control

Model schema

Input and output schema for your model. [Learn more](#)

Name	Type
No schema. See MLflow docs for how to include input and output schema with your model.	

Validate the model before deployment

Run the following code to validate model inference works on the example payload, prior to deploying it to a serving endpoint

```
from mlflow.models import validate_serving_input

model_uri = 'runs:/7410242a778346a690aad5aee9cd9f34/model'

# The logged model does not contain an input_example.
# Manually generate a serving payload to verify your model prior to deployment.
from mlflow.models import convert_input_example_to_serving_input

# Define INPUT_EXAMPLE via assignment with your own input example to the model
# A valid input example is a data instance suitable for pyfunc prediction
serving_payload = convert_input_example_to_serving_input(INPUT_EXAMPLE)

# Validate the serving payload works on the model
validate_serving_input(model_uri, serving_payload)
```

Make Predictions

Predict on a Pandas DataFrame:

Component 4: prediction logging

- Prediction logging records what the model predicted in production
- For each prediction, we should store enough information to understand and debug the result later
- At minimum, prediction logs should include
 - Entity identifier
 - Prediction timestamp
 - Model version
 - Feature version or feature snapshot
 - Prediction value
 - Prediction category or risk band
 - Serving mode
 - Request identifier, if online
 - Scoring workflow execution, if batch
- Prediction logging is essential for debugging, monitoring, auditing, and learning from production behavior
- Prediction logs also allow us to evaluate model performance after deployment, once the true outcomes become available

Component 5: monitoring

- Monitoring tells us whether the ML system is still healthy after deployment
- ML monitoring must include both **system behavior** and **data/product behavior**
- System monitoring asks
 - Did the scoring job run?
 - Did the API respond?
 - How long did it take?
 - How many errors occurred?
 - How many predictions were produced?
- Data and product monitoring asks
 - Are the input features still similar to training data?
 - Are prediction distributions changing?
 - Are some entities missing predictions?
 - Are scores still calibrated?
 - Is model performance degrading?
 - Are consumers receiving fresh outputs?
- This distinction is important as it is not enough to know that the scoring job ran successfully: we also need to know whether the predictions are still meaningful and usable
- A minimum ML monitoring setup should track data drift, prediction drift, model performance decay, scoring freshness, scoring volume, missing predictions, invalid prediction values, latency, failure rate, resource usage...

Component 5: monitoring

- Three monitoring concepts are especially important
 - **Data drift** happens when the input data changes, as when customers now behave differently from the customers used during training
 - **Prediction drift** happens when the model output distribution changes, as when the model suddenly starts assigning high risk to many more customers
 - **Performance decay** happens when the model becomes less accurate or less useful over time. For instance, the churn model had good precision during validation, but after deployment its precision decreases month after month
- These problems are related, but they are not the same
 - A model may have data drift without immediate performance decay
 - A model may also have prediction drift because the population changed, the feature pipeline changed, or the model version changed
- Monitoring should help us detect not only failures, but also changes in model behavior

Component 6: rollback strategy

- A rollback strategy defines what we do when a model or scoring process becomes unsafe
- Rollback may mean
 - Returning to the previous model version
 - Disabling a new model
 - Switching to a baseline model
 - Stopping online predictions
 - Publishing the previous batch scoring table
 - Using a rule-based fallback
 - Blocking publication until validation passes
- Rollback is only possible if previous versions were stored and if serving uses explicit model versions

MLOps in a Data Platform

In practice

Integration with Flyte

- Flyte is a natural bridge between data engineering and ML
- It allows us to describe ML work as a production workflow instead of as a sequence of manual notebook steps
- A training and scoring process can be decomposed into a more or less fixed set of tasks
 1. extract_training_data
 2. validate_training_data
 3. build_features
 4. train_model
 5. evaluate_model
 6. register_model_if_acceptable
 7. batch_score
 8. validate_predictions
 9. publish_prediction_table

Integration with Flyte

1. **extract_training_data**. The first task reads the training data. This task should not simply read “the latest data”. Training data must be selected using explicit intervals, snapshots, and versions. This makes the training run reproducible.
2. **validate_training_data**. This task checks whether the training data is safe to use. Typical checks include schema matches the contract, grain is respected, primary keys are unique, required fields are not null, feature values are within expected ranges, labels are present... This task should fail before training if the data is invalid
3. **build_features**. This task computes the features used by the model. Features may be reusable across multiple models, specific to one model, computed from Silver or Gold tables, materialized for later reuse or generated only for a specific training run. The important point is that feature computation must be deterministic: for the same input data, feature code, and parameters, it should produce the same output.

Integration with Flyte

4. **train_model**. This task trains the model. It should receive explicit inputs: training dataset, feature list, label definition, algorithm, hyperparameters, random seed and train/validation/test split rules. It should produce tracked such as artifacts model file, preprocessing pipeline, feature schema, training metrics, logs and other metadata.
5. **evaluate_model**. After training, the next task evaluates the model. This task compares the model against validation metrics, test metrics, a simple baseline, a previous production model or certain acceptance thresholds. The evaluation task should produce an evaluation report and **a clear decision signal**: a model should only move forward if evaluation shows that it is good enough for the intended use.
6. **register_model_if_acceptable**. The model should only be registered if it passes evaluation. This task promotes the model to a registry state such as candidate, staging, production or archived. If the model does not pass the acceptance criteria, the workflow should stop or mark the model as rejected. Moreover, model registration should be controlled by evidence, not by manual memory.

Integration with Flyte

7. **batch_score**. After a model is registered, it can be used for scoring. This task produces predictions for a logical scoring date. This task should be idempotent: running it twice for the same score date and model version should not duplicate predictions.
8. **validate_predictions**. Before publishing predictions, validate them. Typical checks include expected number of rows, no duplicated keys, no missing required predictions, score values are within valid ranges, prediction distribution is reasonable...
9. **publish_prediction_table**. The final task publishes the validated output. This task writes a stable prediction data product. Publication should happen only after and if validation passes.

Scalable Technologies for Data Analysis

Machine Learning

Davide Carneiro

Mestrado em Engenharia Informática

2025/2026